ILLINOIS

Search in course

Search

English

B

Hide menu

- **Week 11 Information**
- **Introduction to streaming systems**
- **Apache Flink And AWS Kinesis**
- **Spark Streaming**
- **Week 11 Graded Activities**

Graded Assignment: Week 11 Quiz

Submitted

Programming Assignment: Machine Problem 10 V2: Flink & Kinesis

. Duration: 4 hours4h

1. CS 498: Cloud Computing Applications
2. Week 11
3. Machine Problem 10 V2: Flink & Kinesis

PreviousNext

Programming Assignment: Machine Problem 10 V2: Flink & Kinesis

Graded • 3/3 points

Deadline

Pass this assignment by May 4, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview:

You will use a Kinesis data stream as the input source for your Flink application. Your Flink application will perform various technical indicator computations on live stock price data. Based on the computed technical indicators, your Flink application will generate real-time trading signals and alerts. You will deploy and manage your Flink application on AWS Kinesis Managed Flink. You will submit your completed assignment through Coursera.

Exercise A - Build a User Defined Function (UDF) in PyFlink to calculate the compound monthly growth rate (CMGR). Write in Flink SSQL to output data using the UDF CMGR.

Exercise B - Build a 10-day exponential moving average (EMA) that gives more weight to the most recent prices in Flink SSQL to output data below the EMA.

Exercise C - Build an anomaly detection in price changes using MATCH RECOGNIZE in Flink SSQL to output data below the 8% drop percentage.

1.1 Flink Overview:

Apache Flink is a distributed processing engine for big data. It is designed to process data in real-time, using streaming analytics. Apache Flink provides an efficient way to process large volumes of data with low latency and high throughput. It is an open-source platform that is widely used by companies all around the world.

One of the main features of Apache Flink is its ability to process data in real-time. This means that data is processed as soon as it is ingested, allowing for real-time analytics and decision making. Apache Flink also provides support for batch processing, which allows for the processing of large volumes of data in parallel.

Apache Flink provides several APIs for streaming analytics. The most commonly used API is the DataStream API, which provides a high-level abstraction for building streaming applications. The DataStream API allows for the processing of data in a distributed manner, allowing for scalability and fault tolerance.

Another important feature of Apache Flink is its support for stateful stream processing. This allows for the processing of data streams that have stateful operations, such as windowing and aggregation. Apache Flink also provides support for event-time processing, which allows for the processing of data streams based on the time at which events occurred.

Apache Flink is a powerful platform for streaming analytics. Its ability to provide real-time processing of data, support for stateful stream processing, and event-time processing make it an ideal choice for companies looking to process large volumes of data in real-time.

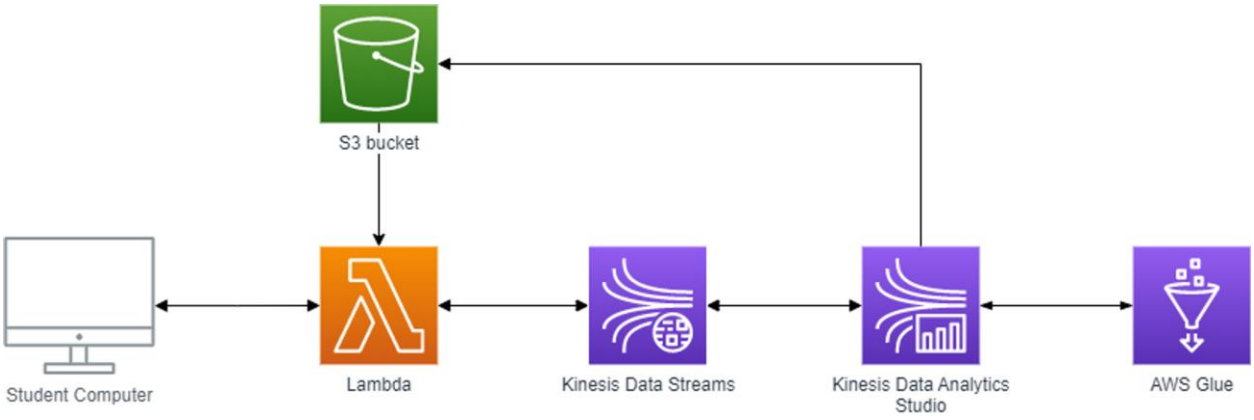
2. General Requirements

To complete this assignment, you will need to use the AWS Kinesis Managed Apache Flink platform and the Apache Flink libraries provided. You will also need to have access to real-time stock market data, which can be obtained by your Kinesis Data Stream where your AWS lambda will parse through the provided csv file and put records to.

Please note that you are not allowed to use any additional libraries or create any files or folders outside of the current folder.

3. Set up the environment

Set up your AWS environment. In your AWS environment you must build a similar architecture as below:



You may refer to this documentation to learn more, https://docs.aws.amazon.com/kinesisanalytics/latest/java/how-notebook.html

4. Procedures

Step 1: Familiarize yourself with Apache Flink and the DataStream API. Read the Apache Flink documentation on the DataStream API.

Step 2: Create an AWS account if you don't already have one.

Step 3: Set up your S3 bucket. Download the provided csv file, and upload it in your S3 bucket. The csv file contains the raw stock price data of AMD that includes **Date, Open, High, Low, Close, Adj Close,** and **Volume** between the year 2021 and 2022.

AMDprices2021-2022

CSV File

Step 4: Set up AWS Kinesis Data Streams in us-east-1. Set the provisioned shard value to 1 to save cost. Your AWS Kinesis Data Streams will take in data from your Lambda function.

Step 5: Set up AWS Lambda. Configure your Lambda functions' permissions with necessary access to S3 and Kineses Data Streams. Your lambda should parse through the csv file stored in your S3 bucket and put the records in your Kinesis Data Streams. Since we don't care about the input, you can run it using the 'Test' feature with anything as the test event data. Make sure the lambda timeout is long enough to read all the data.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

```
import datetime
import json
import boto3
import time
import csv

STREAM_NAME = ""
def lambda_handler(event, context):
    # Initialize S3 client
    s3 = boto3.client('s3')

    # Set bucket name and file key
    bucket_name = ''
    key = ''

    # TODO: Get CSV file from S3
    response = s3.get_object(Bucket=bucket_name, Key=key)
    # TODO: Split data by lines and extract column names from first row
    data = response['Body'].read().decode('utf-8')
    data = data.split('\n')  # Split data by lines
    data_list = []
    columns = data[0].split(',')  # Extract column names from first row
    kinesis_client = boto3.client('kinesis', region_name='us-east-1')
    # TODO: Iterate over rows and generate data for Kinesis stream
    for row in data[1:]:
```

```
    row_data = row.split(',')  # Split row data by comma
    if len(row_data)<len(columns):
        continue
    data_dict = dict(zip(columns, row_data))
    generate(STREAM_NAME, kinesis_client, data_dict)
    time.sleep(0.01)
  # Return response
  return {
    'statusCode': 200,
    'body': json.dumps('Hello from Lambda!')
  }
```

```
def get_data(data_dict):
  # Generate data dictionary for Kinesis stream
```

Step 6: Set up AWS Kinesis Managed Apache Flink in us-east-1. Create a studio notebook and choose 'Create with custom settings'. Your runtime should be “Apache Flink 1.15, Apache Zeppelin 0.10.” Set parallelism and parallelism per KPU to 1, as this will result in minimal costs to your AWS account, and disable VPC connectivity. After creating your notebook, attach kinesis permission to your notebooks' IAM role.

You may refer to this documentation for setting up, https://docs.aws.amazon.com/kinesisanalytics/latest/java/example-notebook-streams.html

To lower costs, make sure to shut down the notebook and Kinesis application when they are not in use, and delete the Kinesis stream when finished. You will be charged for active Kinesis streams regardless of if they receive new data.

To get started on your notebook, create your table

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

```
%flink.ssql(type=update)


-- create a table to glue data catalog table with columns for stock data,
```

```
-- sets a watermark to trigger late arrival events, and configures it to read
-- from a Kinesis stream, in JSON format with an ISO-8601 timestamp format.
CREATE TABLE stock_table(
   `date` STRING,
   ticker VARCHAR(6),
   open_price FLOAT,
   high FLOAT,
   low FLOAT,
   close_price FLOAT,
   adjclose FLOAT,
   volume BIGINT,
   event_time TIMESTAMP(3),
   WATERMARK FOR event_time as event_time
) WITH ( --connect to your kinesis data stream
   'connector' = '',
   'stream' = '',
   'aws.region' = '',
   'scan.stream.initpos' = '',
   'format' = '',
   'json.timestamp-format.standard' = ''
);
```

Exercise A: Compound Monthly Growth Rate

Build a User Defined Function (UDF) in PyFlink to calculate the compound monthly growth rate (CMGR). Write in Flink SSQL to output data using the UDF CMGR.

CMGR=((MonthlyPrice/StartPrice)1/numberofmonths−1)∗100%$CMGR=((MonthlyPrice/StartPrice)1/numberofmonths-1)*100\%$C, M, G, R, equals, left parenthesis, left parenthesis, M, o, n, t, h, l, y, P, r, i, c, e, slash, S, t, a, r, t, P, r, i, c, e, right parenthesis, start superscript, 1,

slash, n, u, m, b, e, r, o, f, m, o, n, t, h, s, end superscript, minus, 1, right parenthesis, times, 100, percent

StartPrice = 92.30 (The first closing price of 2021) Extract year, month, and day values from date string. Check if the day is within the first three days of the month and not January of 2021. Compute the number of months between the start date and the input date. Compute the CMGR*100%. Return 0 if the day is not within the first three days of the month or it is January of 2021. Register the custom function with a name.

In a new Flink SSQL paragraph:

1. Compute CMGR for each row using the registered "cmgr" function
2. Select event time, date, close price, and computed CMGR,
3. Filter out rows where CMGR is 0

The first few expected output should look like the following:

event_time	cmgr	date	close_price
2023-04-28 23:51:38.13	-5.0270815	2/1/2021	87.66
2023-04-28 23:51:38.352	-3.7269766	2/2/2021	88.86
2023-04-28 23:51:38.61	-4.777899	2/3/2021	87.89



The example below builds a UDF in PyFlink to calculate the average of two inputs and outputs the result in Flink SSQL. For continuously re-creating the function, you can add an SSQL paragraph above with the statement 'DROP TEMPORARY FUNCTION IF EXISTS <function_name>' to delete your function.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

```
%flink.pyflink

from pyflink.datastream import StreamExecutionEnvironment
from pyflink.table import StreamTableEnvironment, DataTypes
from pyflink.table.udf import udf



# Define the custom function
```

```
@udf(result_type=DataTypes.FLOAT(), input_types=[DataTypes.FLOAT(), DataTypes.FLOAT()
])
def to_average(low: float, high: float) -> float:
  # Calculate the average of low and high
  avg = (low + high) / 2
  # Return the result
  return avg

# Register the custom function with a name
st_env.create_temporary_function("to_average", to_average)
```

```
%flink.ssql(type=update)

SELECT low, high, event_time, to_average(low, high) AS avg_price
FROM stock_table
```

Once you get the desired output, make sure to cancel your running query/job before runnign next query/job. Otherwise you might get NoResourceAvailableException error.

Exercise B: Exponential Moving Average

Build a modified version of a 10-day exponential moving average (EMA) in Flink SSQL, and output only the entries with a lower close price than the EMA for that entry.

We first calculate the 10-period moving average(simple moving average/SMA) using the **AVG** function, as shown in the example below:

```
1
2
```

3

4

5

6

7

8

9

10

11

12

13

14

15

16

```
%flink.ssql(type=update)

/* stock prices of when the current price is lower than the moving average which would be a
 good indicator for a bear market*/

SELECT *

FROM (

 SELECT

   event_time,

   ticker,

   close_price,

   AVG(close_price) OVER (

     PARTITION BY ticker

     ORDER BY event_time
```

```
    ROWS BETWEEN 9 PRECEDING AND CURRENT ROW
  ) AS avg_close_price
 FROM stock_table
) t
WHERE close_price < avg_close_price;
```

The EMA is then calculated using a modified version of the standard EMA formula, where the natural logarithm (**LN**) of the closing price is first calculated and then summed over a window of 10 days using the **SUM** function. This sum is then divided by 10 to get the average value of the natural logarithms. This average value is then exponentiated using the **EXP** function to get the EMA of the current day.

EMAi=e(∑j=i−10iln(close_pricej))/10*EMAi*=*e*(∑*j*=*i*−10*iln*(*close_pricej*))/10E, M, A, start subscript, i, end subscript, equals, e, start superscript, left parenthesis, sum, start subscript, j, equals, i, minus, 10, end subscript, start superscript, i, end superscript, l, n, left parenthesis, c, l, o, s, e, _, p, r, i, c, e, start subscript, j, end subscript, right parenthesis, right parenthesis, slash, 10, end superscript

LN and **EXP** are mathematical functions that are used to calculate the smoothing constant. **LN** is the natural logarithm function, and **EXP** is the exponential function. In our case, we use **LN** and **EXP** to calculate a smoothing factor based on the number of days we want to use for our EMA.

The first few expected output should look like the following:

ticker	date	close_price	avg_close_price	ema_close_price
AMD	1/15/2021	88.21	92.853004	92.81656214165034
AMD	1/19/2021	89.45	92.56801	92.52590462841277
AMD	1/20/2021	88.75	92.166	92.1169228457768



Exercise C: Anomaly Detection

Build an anomaly detection in price changes using **MATCH RECOGNIZE** in Flink SSQL to output data below the 8% drop percentage. The query should detect drops in stock prices

within 3 observations of a starting point, and return information about those drops including the percentage drop and the initial and final prices.

1

B AS -100*(B.close_price - A.close_price)/A.close_price < 8.0

- The query should start by selecting all columns from your table, and then apply the **MATCH_RECOGNIZE** function.
- The **MEASURES** clause specifies what data to extract from each match, including the time of the initial price, the time of the drop, the percentage drop in price, and the initial and final prices.
- The **ONE ROW PER MATCH** clause ensures that only one row is returned for each match.
- The **AFTER MATCH SKIP PAST LAST ROW** clause is used to skip over rows that have already been matched, so that matches do not overlap.
- The **PATTERN** is defined as a sequence of rows where the first row (A) has the initial price, followed by up to 2 rows (B{0,2}) with prices within 8% of the initial price, and ending with a final row (C) with a price drop of at least 8%.
- The **DEFINE** clause is used to define the condition for the B rows, which is that the price drop is less than 8%.

The first few expected output should look like the following:

ticker	event_time	initialPriceDate	dropDate	dropPercentage	initialPrice	lastPrice
AMD	2024-04-16 22:48:23.65	1/26/2021	1/29/2021	9.576602	94.71	85.64
AMD	2024-04-16 22:48:24.41	3/1/2021	3/4/2021	10.001157	86.39	77.75
AMD	2024-04-16 22:48:28.1	8/4/2021	8/9/2021	9.421567	118.77	107.58

Exercise D: Deploy Flink Application

Until now, your code has been running in a notebook, rather than as a live application. In this exercise you will deploy your code within AWS Managed Flink, allowing it to process data continuously and store its results in S3, rather than simply as a table output. The steps below are fairly precise, but here are some guides that might also be helpful:

https://docs.aws.amazon.com/managed-flink/latest/java/how-notebook-durable.html

https://docs.aws.amazon.com/managed-flink/latest/java/example-notebook-deploy.html

https://docs.aws.amazon.com/managed-flink/latest/java/example-notebook-deploy.html

1. Create a new note(from the Zeppelin home page - no need to make a new studio notebook) in Zeppelin free of any debugging paragraphs you may have added.

2. Create an S3 bucket location for your Flink application code to be stored. Ensure the notebook has an IAM role allowing it to access S3 and the bucket policy allows for it to create and write files there.

3. In the configuration for the studio notebook, edit '*Deploy as application configuration - optional* ' to use this S3 location and allow it to update.

We must now make some code changes to ensure proper function.

4. Copy over the first paragraph, containing the stock_table definition. Change its 'scan.stream.initpos' parameter to LATEST.

5. Add new table definitions to this paragraph to allow for tables that can be stored in S3. Create the bucket/bucket locations you wish to use here as well. You can use the code below:

```
1

2

3

4

5

6

7

8

9

DROP TABLE IF EXISTS part_a;

DROP TABLE IF EXISTS part_b;

DROP TABLE IF EXISTS part_c;

CREATE TABLE part_a(event_time TIMESTAMP(3), cmgr FLOAT, `date` STRING, close_price FLOAT)

WITH ('connector' = 'filesystem','path' = 's3://<bucket_name>/part_a','format' = 'json','json.timestamp-format.standard' = 'ISO-8601');
```

CREATE TABLE part_b(ticker VARCHAR(6), `date` STRING, close_price FLOAT, avg_close_price FLOAT, ema_close_price DOUBLE)

WITH ('connector' = 'filesystem','path' = 's3://<bucket_name>/part_b','format' = 'json','json.timestamp-format.standard' = 'ISO-8601');

CREATE TABLE part_c(ticker VARCHAR(6), event_time TIMESTAMP(3), initialPriceDate STRING, dropDate STRING, dropPercentage DOUBLE, initialPrice FLOAT, lastPrice FLOAT)

WITH ('connector' = 'filesystem','path' = 's3://<bucket_name>/part_c','format' = 'json','json.timestamp-format.standard' = 'ISO-8601');

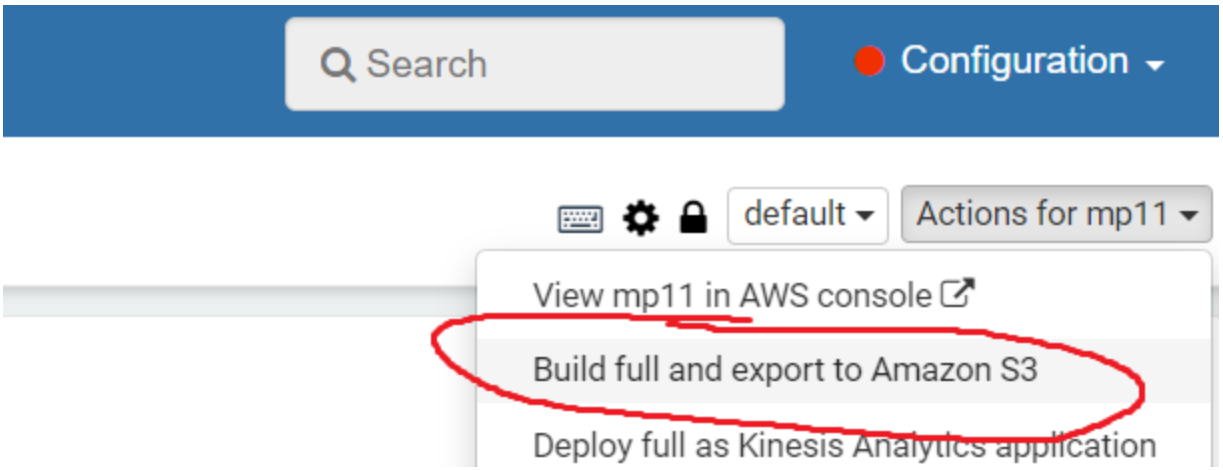
6. In the second paragraph, add your CMGR function definition in Pyflink.

7. In the next paragraph, you will convert all 3 parts of the query into **INSERT INTO** statements in the same paragraph, which will add the output from that query to the tables defined above.

Since this is a streaming application and each part's query can run forever, we must also ensure that all the statements can run at once rather than the default behavior(sequential execution). To enable this, the paragraph should use the **runAsOne** parameter:

```
1
2
3
4
5
6
%flink.ssql(runAsOne=True)
INSERT INTO part_a
  SELECT ...
  ;
INSERT INTO part_b ...;
...
```

8. Ensure your queries are able to run, then build the application. This will output binaries to the S3 location from earlier.



9. After it finishes building, you can deploy the Flink application using the option from the same menu above. After Flink application is deployed, add an IAM role while configuring it with both S3, Glue, and Kinesis permissions for the data locations and stream. Additionally lower the application's parallelism to 1 after creating it to lower costs(the default is 12).

You can now run this application and it will start processing data through all three queries from the stream. We will now set up Lambda functions to test this.

10. Modify the code below to create the first Lambda function('input') which will add data to a Kinesis stream for your application to process. Ensure that it has permissions to write to Kinesis and S3, and a timeout of at least 1 minute. Next, go to 'Configuration -> Function URL' to create a public (unauthenticated) URL for the function.

1

2

3

4

5

6

7

8

9

10

11

```
38
39
40
import json
import boto3
from boto3 import client as boto3_client
import uuid
import time, datetime



# Adds data to Kinesis stream, also clears s3 output data folders beforehand.
# TODO: stream name, folder names and s3 file path
STREAM_NAME = "<your Kinesis stream name>"
bucket_name = '<your s3 bucket for output data>'
folder_names= ['<part A table folder relative to above bucket>','<...>','<...>']
def lambda_handler(event, context):
    for name in folder_names:
        s3 = boto3.resource('s3')
        bucket = s3.Bucket(bucket_name)
        bucket.objects.filter(Prefix=name+'/').delete()
    time.sleep(10)
    data = json.loads(event['body']).splitlines()  # Split data by lines
    print(data)
    columns = data[0].split(',')  # Extract column names from first row
    kinesis_client = boto3.client('kinesis', region_name='us-east-1')
    # TODO: Iterate over rows and generate data for Kinesis stream
```

```
    for row in data[1:]:
        row_data = row.split(',')  # Split row data by comma
        if len(row_data)<len(columns):
            continue
        data_dict = dict(zip(columns, row_data))
        generate(STREAM_NAME, kinesis_client, data_dict)
        time.sleep(0.01)
    # Return response
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
#this allows for part B window to not carry over between runs
def get_data(data_dict):
        # Generate data dictionary for Kinesis stream
    return {
        'date' : data_dict['Date'],
```

11. Modify the code below to create the second Lambda function('output') which will read the output from your Flink application in S3. Ensure it has permissions to read from the bucket and a timeout of at least 1 minute. Next, go to 'Configuration -> Function URL' to create a public URL for the function.

```
1
2
3
4
5
6
```

```
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
import boto3, time, json

# Reads output data tables from S3.
# TODO: folder names and s3 file path
bucket_name = '<your s3 bucket for output data>'
folder_names= ['<part A table folder relative to above bucket>','<...>','<...>']
def lambda_handler(event, context):
```

```
  res = {}
  for i, name in enumerate(folder_names):
    s3_client = boto3.client('s3')
    response = s3_client.list_objects_v2(Bucket=bucket_name, Prefix=name+'/')

    names=[obj['Key'] for obj in response.get('Contents', [])]
    print(names)

    if names:
      modif_data = []
      for file in names:
        s3_response = s3_client.get_object(Bucket=bucket_name, Key=file)
        modif_data.append(s3_response['Body'].read().decode('utf-8'))
      res[i] = '\n'.join(modif_data)
    else:
      print("No data found in S3 folder '%s'" % (name))
      res[i]={}
  return res
```

Now, data fed to the first Lambda function will be added to the Kinesis stream, processed by the live Flink application, and output to S3 locations that will be read when the second lambda is called. We recommend using the autograder script to test this, in addition to the Flink dashboard and logs available through the Managed Flink console.

12. Submission

Download the test.py file attached below to submit your work. You must modify the required parameters within test.py that take in your email address, Coursera secret token, and the two public Lambda URLs corresponding to your functions from part D.

test

PY File

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

bQm7NxnZgRgI2l0p

This token expires in 9 minutes

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue